PLATFORM OPERATIONS GUIDE

Mode-B Agents & Post-Call Webhooks

ElevenLabs Conversational AI setup, secrets, phone calls & platform fixes — Clara AI Platform reference

A complete operator's reference for enabling real-time streaming voice agents on Clara AI Platform: configuring the Mode-B RAG tool secret, registering post-call webhooks in the ElevenLabs dashboard, verifying each step against the live deployment, wiring Twilio phone calls, and applying the language catalogue and sign-out fixes queued for the next release.

Environment: Production

clara.ai.amazetechclans.com

Verified: 18 Sep 2026

CLARA AI PLATFORM

Applies to the ElevenLabs Conversational AI integration (Mode-B, migrations 033–038)

WHAT'S INSIDE

01 **Overview — the two voice tracks** pipeline vs Mode-B agents, live platform state

02 **Mode-B tool secret** what it does, current status, rotation & verification

03 **Post-call webhook secret** dashboard setup, step-by-step

04 **First Mode-B agent & tier limits** prerequisites and account upgrade

05 **Phone calls with Twilio** wiring inbound and outbound calls

06 **Troubleshooting** the 403 origin error, secret errors, restarts

A **Allowed languages — completing the catalogue** 34 missing languages + exact code change

B **Sign-out redirect fix** landing page instead of login

C **Environment & API reference** keys, endpoints, two-step save flow

SECTION 01

Overview — the two voice tracks

Clara AI Platform now runs **two complementary voice architectures** side by side, and they serve different purposes. Understanding which is which avoids most configuration confusion, because the two tracks have different prerequisites, different secrets, and different failure modes. The platform selects the track automatically based on the runtime mode reported by the voice session endpoint.

Track 1 — Pipeline mode (Mode A, already live)

The browser records short audio bursts and sends them to Clara's own REST endpoints: `POST /api/voice/elevenlabs/transcribe` (Scribe STT) → `POST /api/embed/chat` (RAG chat over SSE) → `POST /api/voice/elevenlabs/tts` (speech synthesis, `pcm_16000`). Every turn is a request/response cycle, so latency accumulates per sentence, but no extra secrets are needed. This is the mode used by the chat-window voice chat and by the embed voice demo today.

Track 2 — Mode-B agents (ElevenLabs Conversational AI, this guide)

A true real-time duplex conversation over a single WebSocket (`wss://api.elevenlabs.io/v1/convai/conversation`). The assistant listens, thinks, and speaks with sub-second responsiveness, supports interruptions, and — critically for the platform roadmap — can be attached to a **phone number** so the same agent answers real telephone calls. Because ElevenLabs' servers call back into Clara for knowledge retrieval and call analytics, this track requires the two security secrets this guide configures.

Live platform state (verified 18 Sep 2026)

Every value below was read directly from the production deployment's settings API and the ElevenLabs account status endpoint on 18 September 2026. Treat this table as the baseline: when something misbehaves, compare against this state first.

Item	Status	Notes
ELEVENLABS_ENABLED	true	Voice provider active for chat-window voice and embed voice.
ELEVENLABS_CONVERSATIONAL_AI_ENABLED	true	Mode-B opt-in, set 18 Sep 2026. Documented as requiring a restart for the CSP change; the realtime WebSocket already works because `wss:` is in the CSP connect-src.
ELEVENLABS_TOOL_HMAC_SECRET	set 18 Sep	Mode-B tool secret — configured and verified live during preparation of this guide (see Section 02).
ELEVENLABS_WEBHOOK_SECRET	not set	**Requires the ElevenLabs dashboard value** — follow Section 03.
ELEVENLABS_API_KEY	connected	Account is on the **free tier**: 10,000 characters/month, 11% used, 3 custom voices, instant voice cloning unavailable.
ELEVENLABS_DEFAULT_TTS_MODEL	eleven_multilingual_v2	Platform default; embed pipeline currently resolves to `eleven_flash_v2_5`.
ELEVENLABS_DEFAULT_STT_MODEL	scribe_v1	Scribe STT for both pipeline and agent conversations.

CHANGES APPLIED TO PRODUCTION ON 18 SEP 2026

During diagnosis for this guide, two fixes were applied directly against the live deployment: (1) the **"Landing Demo"** embed API key (`clr_463c2914ca28…`) had an empty `allowed_origins` list, which caused the 403 error on `/api/voice/elevenlabs/session` — the production origin `https://clara.ai.amazetechclans.com` was added and the session endpoint now returns a valid pipeline configuration; (2) the **Mode-B tool secret** was generated (64 hex characters) and stored through the platform's own two-step settings API. No other production data was modified.

SECTION 02

Mode-B tool secret (ELEVENLABS_TOOL_HMAC_SECRET)

When a Mode-B agent is created, Clara registers a **server tool** (webhook tool) on the ElevenLabs agent so that every answer is grounded in your knowledge base. ElevenLabs' agents runtime calls that tool over HTTPS while the conversation is in flight:

The signed RAG callback flow

- User speaks → ElevenLabs agent decides it needs knowledge-base context.
- Agent calls `POST /api/internal/elevenlabs/tools/rag-search` on Clara, carrying an HMAC signature derived from the tool secret that Clara embedded in the agent's tool configuration at creation time.
- Clara verifies the signature, runs hybrid RAG retrieval over the organisation's knowledge base, and returns the context chunk to the agent.
- The agent speaks a grounded answer in real time — no browser round trips involved.

This is why the settings page marks the field as *"Required before any Mode-B agent can be created"*: without the secret, Clara cannot sign the tool configuration, and the internal endpoint refuses every call. The secret never appears in the browser and never needs to be typed into the ElevenLabs dashboard — it stays server-side and is applied automatically when agents are created or re-synced.

CURRENT STATUS — ALREADY CONFIGURED

This secret was set up on 18 September 2026 using the platform's own settings API. It is a 64-character hexadecimal value (256 bits of entropy, above the 32+ character requirement), stored in `platform_env_settings` and masked in the UI. No further action is needed to satisfy the "Required before any Mode-B agent can be created" gate.

How to verify it is live

The internal tool endpoint reports its state through distinct error codes. Probing it with an unsigned request tells you exactly which stage of configuration is missing:

VERIFICATION — UNSIGNED PROBE (SAFE, NO SIDE EFFECTS)

```
curl -s -X POST https://clara.ai.amazetechclans.com/api/internal/elevenlabs/tools/rag-search \ -H "Content-Type: application/json" -d '{"query":"test"}' # Before the secret exists → HTTP 503 {"error":"Tool callback rejected (missing_secret)."} # After the secret exists → HTTP 401 {"error":"Tool callback rejected (missing_signature)."}# 401 missing_signature is EXPECTED for unsigned probes — it proves the secret is live.
```

How to set or rotate it later

1. **Generate a fresh secret**

   Generate 64 random hex characters (well above the 32+ requirement) on any trusted machine: `openssl rand -hex 32`. Avoid reusing values from other services.

2. **Save it through Settings → AI**

   Open `Settings → AI → ElevenLabs → Mode-B tool secret`, paste the value, and save. The UI stores it in `platform_env_settings` via a two-step confirmation flow and immediately masks it.

3. **Re-sync existing agents**

   Rotating the secret invalidates the signature embedded in previously created Mode-B agents. After rotation, re-save or re-sync each agent so Clara re-writes the tool configuration with the new signature, then re-run the verification probe above.

SECTION 03

Post-call webhook secret (ELEVENLABS_WEBHOOK_SECRET)

After every Mode-B conversation ends — web session or phone call — ElevenLabs analyses the call and delivers a full report to your webhook endpoint: transcripts, timestamps, conversation analytics, and metadata. Clara receives these reports at `P-`

`OST /api/webhooks/elevenlabs/post-call` and stores them as conversation records so organisations get call history without any browser involvement.

ElevenLabs signs every delivery with an **HMAC signature header** (`ElevenLabs-Signature`). The signing secret is **generated by the ElevenLabs dashboard when you create the webhook** — that is why this value, unlike the tool secret, must be copied from the dashboard into Clara. Clara additionally guards against replay attacks using a nonce store, so the same delivery can never be accepted twice.

ORDER MATTERS

Create the webhook in the ElevenLabs dashboard **first** — the dashboard generates and reveals the signing secret at that moment — then paste that secret into Clara. Doing it in reverse leaves you with a secret you cannot retrieve later without deleting and recreating the webhook.

Step-by-step setup

1 **Open the ElevenAgents settings page**

Sign in at `elevenlabs.io`, open **Conversational AI / ElevenAgents**, and go to its **Settings** page — direct link: `https://elevenlabs.io/app/agents/settings`. Post-call webhooks are configured workspace-wide here (they apply to all agents, including phone agents).

2 **Register the webhook URL**

In the **Post-call webhooks** section, add Clara's endpoint exactly as:
`https://clara.ai.amazetechclans.com/api/webhooks/elevenlabs/post-call`

3 **Choose the webhook types**

Enable `post_call_transcription` — it carries the full conversation data (transcripts, analysis, metadata) that Clara stores. Optionally add `post_call_audio` (base64 recording — large payloads) and `call_initiation_failure` (failure diagnostics for outbound/batch calls).

4 **Copy the generated signing secret**

When the webhook is created, the dashboard generates and displays its signing secret. Copy it immediately and store it in your password manager — it is the exact value Clara will need.

5 **Paste it into Clara**

Open `Settings → AI → ElevenLabs → Post-call webhook secret`, paste the dashboard value, and save. The field description in Clara ("Register the URL in the ElevenLabs dashboard") refers to steps 1–4 you just completed.

6 **Verify end-to-end**

Run a short test Mode-B conversation, then confirm the conversation appears in Clara's conversation history / call logs. Before setup, an unsigned probe of the webhook endpoint returns `{"error":"Webhook rejected (missing_secret)."}` with HTTP 503; after the secret is stored, unsigned probes are rejected as invalid-signature while genuine signed deliveries succeed.

DELIVERY BEHAVIOUR TO BE AWARE OF

ElevenLabs requires webhook endpoints to answer with HTTP 200. Deliveries that repeatedly fail are **auto-disabled after 10 consecutive failures** (when the last success was more than 7 days ago or there never was one), and failed deliveries are retried automatically. If call history stops updating, check that Clara is reachable and that the stored secret still matches the dashboard value.

SECTION 04

First Mode-B agent & account tier

With both secrets in place, creating a Mode-B agent is a normal platform flow: an organisation opts into Conversational AI for one of its agents (knowledge base), acknowledges the ElevenLabs data-flow disclosure, and Clara provisions the ElevenLabs agent together with its signed RAG tool. The acknowledgement step exists because Mode-B sends conversation audio and transcripts through ElevenLabs' infrastructure — a deliberate, audited decision per organisation rather than a silent default.

Prerequisites checklist

- `ELEVENLABS_CONVERSATIONAL_AI_ENABLED` is **true** (already set).
- Mode-B tool secret configured (already done — Section 02).
- Post-call webhook secret configured (Section 03 — the only outstanding item).
- Restart the Next.js service once after toggling Conversational AI, so the documented CSP update for `wss://api.elevenlabs.io` is applied; realtime websockets already connect because `wss:` is present in the current policy.
- The organisation's knowledge base is indexed and answers questions in pipeline mode.

The free tier will not carry production

The connected ElevenLabs account is currently on the **free tier**: 10,000 TTS characters per month (11% already consumed at verification time), only 3 custom voices, and instant voice cloning disabled. That is adequate for a first proof-of-concept conversation, but a single realistic testing session burns thousands of

characters, and phone agents add their own consumption. Before exposing Mode-B to real users:

Tier	What it unlocks	Recommended for
Free	10k chars/month, 3 voices, no cloning	One-off smoke tests only — current state.
Creator	≈30k chars/month, more voices, usage analytics	Internal pilot with a handful of testers.
Pro	≈100k chars/month, instant cloning, higher concurrency	Production web + phone agents.

Exact quotas and prices change — confirm current limits at `elevenlabs.io/pricing` before committing. Character consumption is monitored on the platform side too: the credit alert threshold (`ELEVENLABS_CREDIT_ALERT_THRESHOLD_PCT`, currently 80%) emails super admins when the monthly budget approaches exhaustion.

SECTION 05

Phone calls with Twilio

Phone calls reuse the exact same Mode-B agents that power web conversations — the agent is simply attached to a phone number, and ElevenLabs bridges telephony audio to the conversation WebSocket. This is why the call centre of the platform's roadmap depends on Sections 02 and 03 being complete: the signed RAG tool keeps phone answers grounded, and the post-call webhook is what records every completed call back into Clara.

Two integration patterns

Pattern	How it works	When to choose
ElevenLabs native Twilio integration	You link your Twilio account (or import a number) in the ElevenLabs dashboard; ElevenLabs manages call media streams and binds the number to an agent.	**Recommended default.** Least code, handles SIP/media, works with agent updates automatically.
Twilio Programmable Voice + agent API	Your Twilio number's webhook tells Twilio to connect the call to the agent; optional "register call" flow for passing caller context.	When you need custom IVR, routing logic, or CRM lookups before the agent picks up.

Platform-side settings

Clara stores telephony credentials per platform and per knowledge base in `phone_call_configs`. The platform-level fields currently sit under **Settings** → **Phone** and are empty today:

- `PHONE_PROVIDER` — set to `twilio` (already configured).
- `TWILIO_ACCOUNT_SID` — your Twilio Account SID (find it on the Twilio console dashboard).
- `TWILIO_AUTH_TOKEN` — the matching auth token.
- `TWILIO_PHONE_NUMBER` — the number callers dial (E.164 format, e.g. `+94771234567`).

Recommended wiring order

1 **Buy / verify a Twilio number**

In the Twilio console, purchase a number with voice capability in the country your callers dial from. For Sri Lanka / India deployments, confirm local number availability or use a regional Twilio subaccount.

2 **Connect Twilio in the ElevenLabs dashboard**

Follow the native integration guide (`elevenlabs.io/docs/eleven-agents/phone-numbers/twilio-integration/native-integration`) to link the account or import the number, then assign the number to your Mode-B agent.

3 **Fill Clara's phone settings**

Enter the SID, auth token, and number under Settings → Phone so the platform can place outbound calls and log per-KB call configuration.

4 **Test inbound + outbound and confirm logs**

Call the number, hold a short grounded conversation, then verify the call appears in Clara's call logs via the post-call webhook. SMS-based conversations and batch outbound calling are available as later extensions.

BILLING REALITY CHECK

Telephony costs are separate from ElevenLabs character quotas: Twilio bills per minute per leg (inbound and outbound), and ElevenLabs bills conversation/character usage on top. Estimate combined per-minute cost before launching any call campaign, and keep the credit alert threshold enabled.

SECTION 06

Troubleshooting

The 403 "Origin is not in the allowed origins" error

This is the error the embed voice demo produced before 18 September. Every embed API key carries an `allowed_origins` list; the voice session endpoint compares the browser's `Origin` header against it and rejects mismatches with HTTP 403. The "Landing Demo" key had an **empty list**, so every origin — including the platform's own domain — was rejected. The key's cryptographic hash itself was valid (verified byte-for-byte), which is why the error message mentioned origins rather than an invalid key.

FIX APPLIED ON 18 SEP 2026 (EQUIVALENT SQL, FOR REFERENCE)

```
-- Add the hosting origin to the embed key's allow-list UPDATE api_keys SET allow
ed_origins = ARRAY['https://clara.ai.amazetechclans.com'] WHERE key_prefix = 'clr
_463c2914ca28'; -- "Landing Demo" key
```

For **other keys** that hit the same error (for example the "Rilan Test KBs" and "Production" keys also have empty origin lists today), either edit the key in the admin embed panel and add the exact origin including scheme and no trailing slash, or run the equivalent SQL above with the right `key_prefix`. A companion error, *"Invalid or revoked embed API key"*, means the key value or its `is_active` flag failed instead — check the `api_keys` row.

Secret-related error codes

Response	Code	Meaning & action
`{"error":"Tool callback rejected (missing_secret)."}`	503	`ELEVENLABS_TOOL_HMAC_SECRET` is empty — set it (Section 02).
`{"error":"Tool callback rejected (missing_signature)."}`	401	Secret is live; caller sent no/invalid signature. Expected for manual probes.
`{"error":"Webhook rejected (missing_secret)."}`	503	`ELEVENLABS_WEBHOOK_SECRET` is empty — complete Section 03.
`{"error":"Origin … is not in the allowed origins for this API key."}`	403	Embed key's origin allow-list is missing the hosting origin — see above.
`{"error":"Invalid or revoked embed API key."}`	403	Key unknown or deactivated — check `api_keys.key_prefix` / `is_active`.

Settings that need a service restart

Setting	Restart?	Why
`ELEVENLABS_CONVERSATIONAL_AI_ENABLED`	yes	Recalculates the Content-Security-Policy at boot (adds the documented `wss://api.elevenlabs.io` entry). Toggled after the last restart, so one restart is still pending.
`ELEVENLABS_TOOL_HMAC_SECRET`	no	Read from `platform_env_settings` at request time — takes effect immediately.
`ELEVENLABS_WEBHOOK_SECRET`	no	Same runtime-read behaviour.
`ELEVENLABS_API_KEY / voice & model defaults`	no	Runtime-read; provider clients pick up changes on next call.

WHERE TO LOOK WHEN AGENTS FAIL MID-CONVERSATION

Check in this order: (1) ElevenLabs usage page — free-tier exhaustion silently ends speech; (2) `/api/admin/elevenlabs/status` — connected flag, tier, character usage, credit alert state; (3) the RAG tool probe from Section 02 — a failing tool makes answers ungrounded rather than failing the call; (4) the agent's conversation log in the ElevenLabs dashboard, which shows tool errors ElevenLabs-side.

APPENDIX A

Allowed languages — completing the catalogue

The **Allowed languages** picker in Settings → AI (platform assistant defaults) currently offers 18 languages. Cross-checked against ElevenLabs' own capability pages on 18 September 2026, the catalogue is missing every language the voice stack can serve beyond those 18. Scribe STT understands 99 languages; the multilingual TTS models speak 29 (v2) to 32 (Flash v2.5) of them. Two sensible tiers of additions follow.

Tier 1 — full voice support (ElevenLabs Flash v2.5 TTS + Scribe STT)

These 16 languages work end-to-end in both voice tracks today — add them first; each is a one-line entry in the platform's language constant:

Code	Language	Code	Language
`nl`	Dutch	`fil`	Filipino
`pl`	Polish	`sv`	Swedish
`bg`	Bulgarian	`ro`	Romanian
`cs`	Czech	`el`	Greek
`fi`	Finnish	`hr`	Croatian
`ms`	Malay	`sk`	Slovak
`da`	Danish	`uk`	Ukrainian
`hu`	Hungarian	`no`	Norwegian

Tier 2 — regional priority (Scribe STT strong; TTS falls back)

The platform already ships Sinhala and Thai — languages Scribe understands but multilingual TTS does not officially list — so STT-strong regional languages are

clearly in scope, especially for the South Asian market the platform serves. These 18 additions follow that precedent:

Code	Language	Code	Language
`bn`	Bengali	`ur`	Urdu
`te`	Telugu	`ml`	Malayalam
`kn`	Kannada	`mr`	Marathi
`gu`	Gujarati	`pa`	Punjabi
`ne`	Nepali	`or`	Odia
`fa`	Persian	`ps`	Pashto
`sw`	Swahili	`my`	Burmese
`km`	Khmer	`lo`	Lao
`yue`	Cantonese	`sr`	Serbian

The exact code change

The list lives in the AI settings options module that also exports the chat/embedding/realtime model catalogues — the exported constant rendered as `"English (en)"` style options in the picker. Extend the array with the 34 entries above (full block ready to paste):

LANGUAGES CONSTANT — REPLACEMENT VALUE (34 ADDITIONS APPENDED)

```
export const LANGUAGES = [ { code: "en", label: "English" }, { code: "si", label: "Sinhala" }, { code: "ta", label: "Tamil" }, { code: "hi", label: "Hindi" }, { code: "ar", label: "Arabic" }, { code: "zh", label: "Chinese" }, { code: "ja", label: "Japanese" }, { code: "ko", label: "Korean" }, { code: "es", label: "Spanish" }, { code: "fr", label: "French" }, { code: "de", label: "German" }, { code: "pt", label: "Portuguese" }, { code: "ru", label: "Russian" }, { code: "it", label: "Italian" }, { code: "id", label: "Indonesian" }, { code: "tr", label: "Turkish" }, { code: "vi", label: "Vietnamese" }, { code: "th", label: "Thai" }, // — Tier 1: ElevenLabs TTS-supported ———————————————— { code: "nl", label: "Dutch" }, { code: "fil", label: "Filipino" }, { code: "pl", label: "Polish" }, { code: "sv", label: "Swedish" }, { code: "bg", label: "Bulgarian" }, { code: "ro", label: "Romanian" }, { code: "cs", label: "Czech" }, { code: "el", label: "Greek" }, { code: "fi", label: "Finnish" }, { code: "hr", label: "Croatian" }, { code: "ms", label: "Malay" }, { code: "sk", label: "Slovak" }, { code: "da", label: "Danish" }, { code: "uk", label: "Ukrainian" }, { code: "hu", label: "Hungarian" }, { code: "no", label: "Norwegian" }, // — Tier 2: regional priority (Scribe STT) ———————————————— { code: "bn", label: "Bengali" }, { code: "ur", label: "Urdu" }, { code: "te", label: "Telugu" }, { code: "ml", label: "Malayalam" }, { code: "kn", label: "Kannada" }, { code: "mr", label: "Marathi" }, { code: "gu", label: "Gujarati" }, { code: "pa", label: "Punjabi" }, { code: "ne", label: "Nepali" }, { code: "or", label: "Odia" }, { code: "fa", label: "Persian" }, { code: "ps", label: "Pashto" }, { code: "sw", label: "Swahili" }, { code: "my", label: "Burmese" }, { code: "km", label: "Khmer" }, { code: "lo", label: "Lao" }, { code: "yue", label: "Cantonese" }, { code: "sr", label: "Serbian" }, ]; // After editing: commit, rebuild, redeploy — the picker then shows 52 languages.
```

INTERIM OPTION WHILE THE NEXT DEPLOY IS PENDING

The picker already accepts custom codes: any ISO code stored in `platform_assistant_defaults.allowed_languages` renders as an extra (uppercase) option even before a code change. Current platform defaults are `en`, `ta`, `ar`. The clean fix is the constant above — it is queued with the sign-out change in Appendix B for the next repository deployment.

APPENDIX B

Sign-out redirect fix

Signing out currently sends users to the login screen. The intended behaviour is a return to the public landing page. The sign-out handler lives in the workspace shell component that renders the profile menu (the same component builds the admin and org navigation). Its handler currently reads:

CURRENT HANDLER (MINIFIED IN PRODUCTION BUILD)

```
async function handleSignOut() { try { await logout(); } finally { router.push("/login"); } // ← change this line }
```

FIX — SEND SIGNED-OUT USERS TO THE LANDING PAGE

```
async function handleSignOut() { try { await logout(); } finally { router.push("/"); } // ← landing page }
```

Two details matter when applying this change. First, the same component also contains an *authentication guard* that redirects unauthenticated visitors away from protected pages — that guard (`router.replace("/login")` in a `useEffect`) must remain untouched, otherwise unauthenticated users could linger on protected shells. Second, if the org-facing workspace uses a second copy of the profile menu, apply the same one-line change there so both shells behave identically. Like Appendix A, this ships with the next code deployment.

APPENDIX C

Environment & API reference

ElevenLabs-related settings keys

Key	Secret	Purpose
`ELEVENLABS_API_KEY`	yes	Account key used for all ElevenLabs API calls.
`ELEVENLABS_ENABLED`	no	Master switch for the ElevenLabs voice provider.
`ELEVENLABS_CONVERSATIONAL_AI_ENABLED`	no	Mode-B opt-in; restart to apply CSP change.
`ELEVENLABS_TOOL_HMAC_SECRET`	yes	Signs the RAG server-tool callback (Section 02).
`ELEVENLABS_WEBHOOK_SECRET`	yes	Verifies post-call webhook deliveries (Section 03).
`ELEVENLABS_DEFAULT_TTS_MODEL / _STT_MODEL`	no	Platform default models (multilingual v2 / Scribe v1).
`ELEVENLABS_DEFAULT_VOICE_ID`	no	Fallback voice when orgs pick none.
`ELEVENLABS_VOICE_CLONING_ENABLED / _VOICE_DESIGN_ENABLED`	no	Feature toggles for cloning and voice design.
`ELEVENLABS_CREDIT_ALERT_THRESHOLD_PCT`	no	Email alert when this % of the character budget is consumed.
`TWILIO_ACCOUNT_SID / _AUTH_TOKEN / _PHONE_NUMBER`	mixed	Telephony credentials for phone calls (Section 05).

Key endpoints

Endpoint	Auth	Role
`POST /api/voice/elevenlabs/session`	embed key	Creates a voice session config (pipeline or agent) for the embed demo; enforces the origin allow-list.
`POST /api/voice/elevenlabs/transcribe`	embed key	Scribe STT for pipeline mode.
`POST /api/voice/elevenlabs/tts`	embed key	Speech synthesis for pipeline mode.
`POST /api/internal/elevenlabs/tools/rag-search`	HMAC	Signed RAG tool that ElevenLabs agents call mid-conversation.
`POST /api/webhooks/elevenlabs/post-call`	HMAC	Receives signed post-call reports from ElevenLabs.
`GET /api/admin/elevenlabs/status`	admin	Connection state, tier, character usage, feature flags.
`GET|POST /api/platform/env`	admin	Reads / two-step-writes all runtime settings incl. both secrets.

How settings are saved (the two-step flow)

Every settings save in the UI — including both secrets — uses a confirmation-token flow that prevents cross-site request forgery. The browser first asks the API to mint a short-lived token bound to the new value, then commits the value by presenting that token:

TWO-STEP SAVE (WHAT THE SETTINGS UI DOES INTERNALLY)

```
# Step 1 — mint a confirmation token (valid 5 minutes) GET /api/platform/env?toke
n=1&key=ELEVENLABS_WEBHOOK_SECRET&value=<secret> → { "confirmation_token": "…",
"timestamp": 1789762372296, "expires_in_ms": 300000 } # Step 2 — commit POST /ap
i/platform/env { "key": "ELEVENLABS_WEBHOOK_SECRET", "value": "<secret>", "confir
mation_token": "…", "timestamp": 1789762372296 } → { "ok": true, "key": "ELEVENLA
BS_WEBHOOK_SECRET", "updatedAt": "…" }
```

SECURITY HYGIENE FOR OPERATORS

Secrets stored here are readable in plaintext from the database by anyone with SQL access — protect database credentials accordingly and rotate them on any suspicion of exposure. Prefer generating fresh secrets over reusing values from other tools, never paste secret values into chat tickets or emails, and treat the masked display in Settings → AI as the only appropriate place to confirm a secret's presence. The default admin account password should also be rotated before production hardening is considered complete.

Official documentation referenced in this guide

- Post-call webhooks — `elevenlabs.io/docs/eleven-agents/workflows/post-call-webhooks`
- Webhook tools (server tools) — `elevenlabs.io/docs/eleven-agents/customization/tools/webhook-tools`
- Twilio native integration — `elevenlabs.io/docs/eleven-agents/phone-numbers/twilio-integration/native-integration`
- Twilio API integration — `elevenlabs.io/docs/eleven-agents/customization/integrations/twilio`
- Scribe STT supported languages — `elevenlabs.io/docs/overview/capabilities/speech-to-text`
- TTS supported languages (v2 / Flash v2.5) — `elevenlabs.io/docs/overview/capabilities/text-to-speech`
- ElevenAgents settings dashboard — `elevenlabs.io/app/agents/settings`
- Tiers and quotas — `elevenlabs.io/pricing`

This guide reflects the live deployment as verified on 18 September 2026. The two production changes described in Section 01 (embed key origin allow-list, Mode-B tool secret) were applied through the platform's own mechanisms and require no further action; the remaining items — the webhook secret, an account tier decision, Twilio credentials, and the two code-level fixes in Appendices A and B — are ordered by dependency so each unlocks the next capability of the real-time voice track.

QUICK COMPLETION CHECKLIST

You are done when…

Work through this checklist after reading the guide. Every item below is directly verifiable against the production deployment, and each one unblocks a concrete capability of the platform's real-time voice track.

- ✓ The embed voice demo opens a session without a 403 error (origin fix verified on 18 Sep 2026).
- ✓ Settings → AI shows Mode-B tool secret as configured (set on 18 Sep 2026 via the platform env API).
- ✓ The ElevenLabs dashboard shows the post-call webhook URL pointing at Clara's webhook endpoint.
- ✓ Settings → AI shows the post-call webhook secret as configured (paste from the ElevenLabs dashboard).
- ✓ A first Mode-B agent answers a live voice conversation with grounded RAG answers.
- ✓ The ElevenLabs account is upgraded from the free tier before production traffic.
- ✓ Phone call settings (Twilio SID, token, number) are filled in Settings → Phone.